Understanding logrotate on Ubuntu part 1

It's no fun when log files grow out of control. In this two-part series, learn how to use logrotate to keep those logs in check.

What is logrotate?

It may surprise you to learn that logrotate is a program used to rotate logs. It's true! The system usually runs logrotate once a day, and when it runs it checks rules that can be customized on a per-directory or per-log basis.

"Log rotation" refers to the practice of archiving an application's current log, starting a fresh log, and deleting older logs. And while we're explaining things, a "log" is a file where an application stores information that might be useful to an administrator or developer - what it's been doing, what errors it's run into, that sort of thing. So logs are good, you just usually don't want to keep a ton of them around. That's where logrotate comes in.

The importance of log rotation

Logs are wonderful things when you want to track usage or troubleshoot an application. Unfortunately the more information that gets logged, the more disk space the log uses. Over time it can really add up.

A log left unrotated can grow to a pretty unwieldy size. Running out of disk space because of a giant log is a problem of course, but a huge log file can also slow down the process of resizing or backing up your virtual server. Another practical consideration is that it's hard to look for a particular event if you have a million log entries to skim through. So on the whole it's a good idea to keep log files down to a manageable size, and to prune them when they get too old to be of much use.

Fortunately logrotate makes log rotation easy.

How it works

The system runs logrotate on a schedule, usually daily. In fact, you'll find the script that runs logrotate daily at:

```
/etc/cron.daily/logrotate
```

If you want logrotate to run more often (for hourly log rotation, for example) you'll need to look into using cron to run logrotate through a script in /etc/cron.hourly.

When logrotate runs it reads its configuration files to determine where to find the log files it needs to rotate, and to check on details like how often the files should be rotated and how many archived logs to keep.

logrotate.conf

The main logrotate configuration file is located at:

```
/etc/logrotate.conf
```

If you look inside that file you'll see the default parameters logrotate uses when it rotates logs. The file is nicely commented, so skim it to see how things are set up. We'll talk about several of the specific commands in that file shortly.

Note that one line reads:

```
include /etc/logrotate.d
```

That's where we'll find most of the application-specific configuration files.

logrotate.d

Take a look inside the directory where you'll store application-specific log settings:

```
ls /etc/logrotate.d
```

Depending on how much you've installed on your server there may be no files in this directory, or there may be several. In general, applications that are installed through Ubuntu's package manager (aptitude) will also create a config file in /etc/logrotate.d.

Most likely you will at least see a config file for rsyslog, which logrotate will read when it goes to rotate the system logs. If you look inside you'll see an entry for various system logs along with some commands similar to what you saw in logrotate.conf.

NOTE: You won't actually see an entry for rsyslog on versions of Ubuntu older than Karmic Koala (9.10). Prior to that release the system logs were rotated by a "savelog" command run from the "/etc/cron.daily/sysklogd" script.

Inside an application file

As an example, let's take a look at the contents of a logrotate config file that might be put in place when you install apache:

```
/var/log/apache2/*.log {
    weekly
    missingok
    rotate 52
    compress
    delaycompress
    notifempty
    create 640 root adm
    sharedscripts
    postrotate
        if [ -f "`. /etc/apache2/envvars ; echo ${APACHE_PID_FILE:-/var/run/apache2.pid}`" ]; then
            /etc/init.d/apache2 reload > /dev/null
        fi
    endscript
}
```

We'll look at what most of the specific directives in this file mean in a bit, but for now I'll sum it up, just to provide a frame of reference.

When it goes to rotate the apache logs, logrotate will check for any files in /var/log/apache2 that end in ".log", then rotate them if it's been a week since their last rotation. It will keep 52 archived copies of a log, deleting the oldest log if it goes over that number. After rotating a log a replacement, empty log file will be created. Once logrotate is done checking all the logs in that directory it will run the command in the "postrotate/endscript" block (in this case, a command that will tell apache to gracefully

restart if it's running).

This is a pretty thorough configuration file, so it's worth noting what happens if one of those directives were left out. Back in the logrotate.conf file you may remember seeing a "weekly" command there too, a smaller number for the "rotate" value, and a couple other similar commands. If any of those commands hadn't been specified in the apache-specific config file, those settings would have been inherited from the main logrotate.conf file. For example, the "rotate 52" takes precedence over the "rotate 4" default, since it was specified in a logfile config block.

Didn't get all that? No problem. It should be more clear if we talk about some of those commands in-depth. So let's do that next.

Configuration commands

You can get a full list of commands used in logrotate configuration files by checking the man page:

```
man logrotate
```

We'll go over more commonly-used commands here.

Remember, the config files for applications in /etc/logrotate.d inherit their defaults from the main /etc/logrotate.conf file.

Log files

A log file and its rotation behavior is defined by listing the log file (or files) followed by curly brackets. Most application configuration files will contain just one of these blocks, but it's possible to put more than one in a file, or to add log file blocks to the main logrotate.conf file.

You can list more than one log file for a block either by using a wildcard in the name or by separating log files in the list with spaces. For example, to specify all files in the directory /var/foo that end in ".log", as well as the file "/var/bar/log.txt", you would set up the block like so:

```
/var/foo/*.log /var/bar/log.txt {
    blah blah blah
    blah blah blah redux
}
```

Just not with as many blahs.

Rotate count

The "rotate" command determines how many archived logs will be kept around before logrotate starts deleting the older ones. For example:

```
rotate 4
```

That command tells logrotate to keep 4 archived logs at a time. If there are already four archived logs when the log is rotated again, the oldest one (the one with ".4" at the end, usually) will be deleted to make room for the new archive.

Rotation interval

You can specify a command that will tell logrotate how often to rotate a particular

log. The possible commands include:

daily
weekly
monthly
yearly

If a rotation interval is not specified the log will be rotated whenever logrotate runs (unless another condition like "size" has been set).

If you want to use a time interval other than the keywords listed here you'll have to get clever with cron and a separate config file. For example, if you wanted to rotate a particular log file hourly, you could create a file in "/etc/cron.hourly" (you may need to create that directory too) that would contain a line like:

```
/usr/sbin/logrotate /etc/logrotate.hourly.conf
```

Then put the configuration for that hourly run of logrotate (the log file location, whether or not to compress old files, and so on) into "/etc/logrotate.hourly.conf".

Size

You can specify a file size that logrotate will check when determining whether or not to perform a rotation by using the "size" command. The format of the command tells logrotate what units you're using to specify the size:

```
size 100k
size 100M
size 100G
```

The first example would rotate the log if it gets larger than 100 kilobytes, the second if it's larger than 100 megabytes, and the third if it's over 100 gigabytes. I don't recommend using a limit of 100G, mind you, the example just got a little out of hand there.

The size command takes priority over a rotation interval. When a log rotates because it hit its size limit the time interval resets for that log file. For example, if a log set to rotate "weekly" is rotated because it hit its max size after four days, logrotate will wait another week after that before rotating the log based on the time interval that has passed. As a result, if you use an interval other than "daily" and specify a maximum size for the log file, the log rotation won't be guaranteed to happen on the same day every week.

Compression

If you want archived logfiles to be compressed (in gzip format) you can include the following command, usually in /etc/logrotate.conf:

```
compress
```

This is normally a good idea, since log files are usually all text, and text compresses very well. You might, however, have some archived logs you don't want compressed, but still want compression to be on by default. In those cases you can include the following command in an application-specific config:

```
nocompress
```

One more command of note in regard to compression is:

```
delaycompress
```

This command can be useful if you want the archived logs to be compressed, but not right away. With "delaycompress" active an archived log won't be compressed until the next time the log is rotated. This can be important when you have a program that might still write to its old logfile for a time after a fresh one is rotated in. Note that "delaycompress" only works if you also have "compress" in your config.

An example of a good time to use delaycompress would be when logrotate is told to restart apache with the "graceful" or "reload" directive. Since old apache processes would not be killed until their connections are finished, they could potentially try to log more items to the old file for some time after the restart. Delaying the compression ensures that you won't lose those extra log entries when the logs are rotated.

Postrotate

The "postrotate" script is run by logrotate each time it rotates a log specified in a config block. You'll usually want to use this to restart an application after the log rotation, so the app can switch to a new log.

```
postrotate
   /usr/sbin/apache2ctl restart > /dev/null
endscript
```

That "> /dev/null" bit at the end tells logrotate to pipe the command's output to, well, nowhere. Otherwise the output of that command will be sent off to the console or the log or email or whatever, and in this case, you don't really care about the output if everything restarted okay.

The "postrotate" command tells logrotate that the script to run will start on the next line, and the "endscript" command says that the script is done.

Sharedscripts

Normally logrotate will run the "postrotate" script every time it rotates a log. This is true for multiple logs using the same config block. So for example, a web server config block that refers to both the access log and the error log will, if it rotates both, run the "postrotate" script twice (once for each file rotated). So if both files are rotated, the web server will be restarted twice.

To keep logrotate from running that script for every log, you can include the command:

```
sharedscripts
```

That tells logrotate to wait until it's checked all the logs for that config block before running the postrotate script. If one or both of the logs get rotated, the postrotate script still only gets run once. If none of the logs get rotated, the postrotate script won't run at all.

Summary

You've seen an overview of what logrotate does and what kind of configuration options are available to you. You should be all set to go poking around in the existing configs and adapt them to your needs. But let's not stop there! In the next article we'll look at putting an example config together (to rotate the logs for custom virtual hosts), and also cover some handy troubleshooting approaches.